

Group-2 Project Proposal

CSCN8020 Project Proposal Report

Topic: From A3 DQN left-elbow control to a simple Unitree G1 reach-and-touch virtual target task.

This proposal describes both the A3 reinforcement learning prototype and the plan to implement the final project. The A3 baseline trained a Deep Q-Network (DQN) to control the Unitree G1 robot's left elbow in MuJoCo. The final project extends that baseline from single-joint target control toward a simple reach-and-touch virtual target task, using clear mathematical, algorithmic, and coding evidence.

1. Project Overview in the Context of Reinforcement Learning

The robot is treated as an RL agent. At each time step, it observes the current robot state, selects an action, receives a reward, and transitions to a new state inside the MuJoCo simulation. In A3, the state was a four-value vector: current elbow angle, current elbow velocity, goal angle, and the error between the goal and the current elbow angle. The action space was discrete with three choices: decrease the controller target, hold it, or increase it.

RL component	A3 implementation	Final project extension
Agent	DQN policy controlling the left elbow.	Reach-and-touch policy controlling arm targets.
State	Elbow angle, velocity, goal, and error.	Joint angles, velocities, hand/wrist position, target error, and touch phase.
Action	Three discrete actions: decrease, hold, increase.	Discrete target changes for shoulder and elbow phases.
Reward	Penalizes error; rewards success region and termination.	Rewards target reaching, touch/hold success, and stable motion.
Environment	Unitree G1 Gymnasium environment in MuJoCo.	Extended G1 arm-control environment for touching a virtual target zone.

2. A3 Baseline and Results

A3 implemented a student-written PyTorch DQN with an online Q-network, a target Q-network, replay memory, epsilon-greedy exploration, Bellman target updates, Huber loss, and Adam optimization. Two epsilon-decay configurations were trained for 500 episodes and evaluated greedily on four benchmark target angles.

Metric	Config A	Config B	Interpretation
Episodes	500	500	Both configurations used the same training budget.
Evaluation successes	20 / 20	20 / 20	Both learned reliable benchmark target control.
Evaluation success rate	100%	100%	Meets the A3 success requirement.
Mean evaluation reward	13.282	13.167	Config A was slightly higher.
Mean final absolute error	0.00491 rad	0.00950 rad	Config A had lower final error.

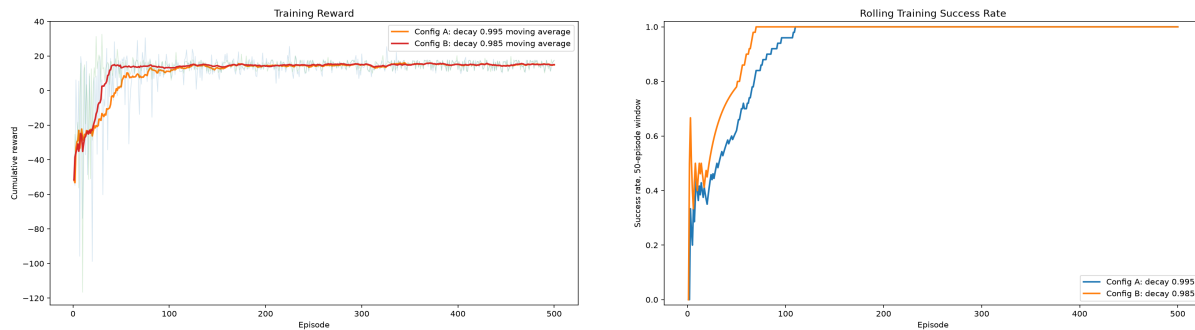


Figure evidence: cumulative reward improves during training, and rolling success rate reaches 1.0 for the trained DQN configurations.

3. Mathematical and Algorithmic Design

Policy, value, and discounting

The policy maps the current state to an action. In notation, $\pi(s)$ chooses the action that the Q-network estimates to have the highest value. The DQN estimates $Q(s,a)$, the expected discounted return for taking action a in state s and then following the learned policy. The discount factor γ was set to 0.95, so the agent values immediate accuracy while still considering future task completion.

Discounted return: $G_t = \sum_{k=t+1}^T \gamma^{k-t-1} R_k$
 Learning objective: maximize over policy π the expected return $E[G_t]$
 Bellman target: $y = r + (1 - \text{done}) * \gamma * \max_{a'} Q_{\text{target}}(s', a')$

DQN vs Actor-Critic

DQN is a strong baseline because the A3 action space is discrete. It selects one of three actions and learns values for those choices. Actor-Critic methods, such as PPO, are often better suited when the action space is continuous because an actor can output smoother control commands while a critic estimates value. For the final project, the first implementation will keep DQN by representing the touch-target task as discrete reach, touch/hold, and return phases. If the motion looks too jerky, Actor-Critic/PPO will be the planned comparison or extension.

4. DQN Learning Cycle: Math, Algorithm, and Code Mapping

Following the instructor's DQN cycle, the A3 implementation can be described as a direct mapping from mathematical terms to algorithm steps and code/log outputs. The neural network approximates $Q(s,a)$, the action is selected by argmax or epsilon-greedy exploration, the environment returns r and s' , and the network is updated by minimizing the Bellman prediction error.

Math term	Algorithm meaning	A3 code or log evidence
s	Current state observed by the agent.	state = [elbow_angle, velocity, goal, error]
$Q(s,a)$	Neural network output for every action.	Example log: $Q(s,a) = [-0.82, -0.45, 0.31]$
$\text{argmax}_a Q(s,a)$	Choose the action with the highest predicted value.	action = 2 # increase target
r and s'	MuJoCo environment returns reward and next state.	reward $r = 0.91$; next_state $s' = [0.46, 0.02, 0.50, 0.04]$
$\max_{a'} Q(s',a')$	Estimate the best future return from the next state.	$Q_{\text{target}}(s',a') = [-0.60, -0.22, 0.40]$; max = 0.40
$y = r + \gamma \max_{a'} Q(s',a')$	Bellman target used for supervised update.	$y = 0.91 + 0.95 * 0.40 = 1.29$
Loss = $(y - Q(s,a))^2$	Prediction error minimized by gradient descent.	loss = $(1.29 - 0.31)^2$; backprop updates the network.

The console output should therefore show state values, Q-values, chosen action, reward, next state, Bellman target, and loss. For the final reach-and-touch task, the same learning cycle remains. The state changes from elbow error to hand-to-target error, the action changes from elbow target steps to shoulder/elbow target steps, and the reward changes from angle accuracy to target reaching, touch holding, and smoothness.

5. Code-Level Evidence

```
Observation = [
    current_elbow_angle,
    current_elbow_velocity,
    goal_angle,
    goal_angle - current_elbow_angle,
]

Actions:
0 = decrease target
1 = hold target
2 = increase target

reward = -absolute_error
if success: reward += 1.0
if terminated: reward += 10.0
```

```
QNetwork:
Linear(4, 64) -> ReLU
Linear(64, 64) -> ReLU
Linear(64, 3)

Action selection:
if random < epsilon:
    explore random action
else:
    choose argmax Q(s, a)

Bellman update:
current_q <- Q_online(s, a)
target_q <- r + gamma * max Q_target(s', a')
```

These snippets show the key coding perspectives: state/action definition, reward design, and the learning update used by the DQN agent.

6. Final Project Plan

The final project will extend the A3 prototype from reaching one elbow target to performing a short reach-and-touch sequence: rest, reach the virtual target, touch and hold the target zone, and return to a neutral position. The goal is not only visual motion, but a measurable learned behavior that can be evaluated through reward, success rate, tracking error, and motion smoothness.

```
Planned reward form: r_t = -||p_hand - p_target||_2 + b_touch - lambda * smoothness_penalty.
This connects the final project to the same return objective used in A3: the robot should maximize the discounted sum of future rewards.
```

Workstream	Plan
Environment	Extend the G1 environment to include shoulder and elbow targets for reach, touch/hold, and return phases.
Action design	Start with discrete target changes so the DQN baseline remains compatible.
Reward design	Reward closeness to the virtual target, touch/hold duration, stable torso, and smooth motion; penalize missed targets or unstable movement.
Training	Train DQN on the reach-and-touch task, tune gamma, epsilon decay, and reward weights.
Comparison	If DQN motion is not smooth enough, compare with an Actor-Critic/PPO approach.
Deliverables	Final curves, evaluation table, saved checkpoint, and a short MuJoCo video of the learned target touch.

7. Evaluation Criteria for the Final Project

- Success rate: whether the hand/wrist endpoint reaches the virtual target and holds the target region.
- Average cumulative reward: whether learning improves over training.
- Final pose and tracking error: how close the hand/arm is to the intended target positions.
- Smoothness: whether the learned motion avoids unnecessary oscillation or unstable jerks.
- Hold duration and episode length: whether the reach-and-touch task can be completed efficiently and consistently.

Sources Used

Local A3 repository sources: README.md; src/g1_rl/g1_elbow_env.py; src/dqn/agent.py; src/dqn/q_network.py; results/plots/config_comparison_summary.csv; results/plots/final_evaluation_table.csv; training_reward.png; rolling_success_rate.png.